Final Exam (Part 1)

Program Design and Data Structures (1DL201)

Teachers: Eva Darulova and Johannes Borgström

2024-01-11 / 14:00 - 18:00

Instructions

Read and follow these instructions carefully to increase your chance of getting good marks.

- This is a closed-book exam. You may use a standard English dictionary. Otherwise, **no notes, calculators, mobile phones, or other electronic devices are allowed**.
- Write your **exam code** at the top right corner of **each page**.
- There are **16 questions**. Each question awards a number of points, as listed in the question heading. You can obtain a maximum of **40 points**.
- Some questions ask for a justification of your answer; your justification needs to relate the information given in the question, the course content, and your answer.
- Some questions ask you to write code; this code needs to conform to the coding convention. You only need to write function documentation if the question text states so.
- Write your answers directly on the question sheet. You may use English and Swedish.
- Eva and/or Johannes will come to the main exam hall around 15:00 to answer questions (and a bit later to other exam locations).

Question 1: Substitution model (2 points)

What is the result of the following program?

```
function add(x, y) {
  return x + y;
}
const input = 3;
add(input, 3 * input);
```

Answer: 12

Show all evaluation steps using the **substitution model**:

```
add(3, 3 * 3);   // substitute input = 3;
add(3, 9);   // evaluate arguments
3 + 9;   // apply function by substitution
12; // evaluate arithmetic
```

Question 2: Recursive functions (3 points)

Write a recursive function that calculates the sum of the digits in the base-10 representation of a non-negative integer (tvärsumman).

Hint: Recall that the `%` operator computes the remainder when its left argument is divided by the argument on the right.

Answer:

```
function digit_sum(n) {
  return n === 0 ? 0 : (n % 10) + digit_sum(math_floor(n / 10));
}
```

Does your function generate an ~~inductive~~ iterative process or not? Why?

Answer: No, because the addition is delayed until the recursive call returns.

Question 3: Orders of growth (2 points)

Which of the following four statements are true?

1. $n \in \mathrm{O}(n^2)$
2. $n^2 \in \Omega(n \log n)$
3. $2^n \in \Theta(n\, 2^n)$
4. $n \in \mathrm{O}(2^n)$

Answer: 1, 2, and 4.

Question 4: Solving recurrences (3 points)

Recall the Master theorem (for divide-and-conquer recurrences):
Assume that $T(n) = a\,T(n/b) + f(n)$ for some $a > 0$, $b > 1$, and non-negative f. We let $E = \frac{\log a}{\log b}$.

1. If there is ε such that $f(n) \in \mathrm{O}\left(n^{E-\varepsilon}\right)$ then $T(n) \in \Theta\left(n^{E}\right)$.
2. If there is k such that $f(n) \in \Theta\left(n^{E}\,(\log n)^{k}\right)$ then $T(n) \in \Theta\left(n^{E}\,(\log n)^{k+1}\right)$.
3. If there is ε such that $f(n) \in \Omega\left(n^{E+\varepsilon}\right)$ and f satisfies the regularity condition (there is $c > a$ and n_0 such that for all $n > n_0$, $f(n) \geq c\,f(n/b)$) then $T(n) \in \Theta(f(n))$.

Use this theorem to solve the recurrence $T(n) = 8\,T(n/8) + n^2$. Show all steps of your solution.

Answer: Here $a = b = 8$ so $E = 1$, $f(n) \in \Omega(n^{E+\varepsilon})$ for $\varepsilon = 1 > 0$ and $n^2 \geq c\,(n/8)^2 = n^2/4$ for $c = 16 \geq 8$, so the work function is regular and case 3 applies (the root dominates). Then $T(n) \in \Theta(f(n)) = \Theta(n^2)$.

Note: in the course HT 2025, we do not prove the regularity condition in HT 2025.

Question 5: Higher-order functions (3 points)

We aim to automate the grading system for the autumn homework module. There are three homeworks and each is graded on a scale of 0 to 100 points. A homework is passed if its points are 50 or above. A grade of "G" is given for the whole module if all three homeworks are passed and the average score is from 50 to 74, and a grade "VG" for an average score 75 and above. Write two **lambda expressions** `is_passing` and `average_grade` that can be used as follows:

```
function grade_student(pass_func, avrg_func, score1, score2, score3) {
    if(pass_func(score1) && pass_func(score2) && pass_func(score3)) {
        return avrg_func(score1, score2, score3);
    } else {
        return "U";
    }
}
grade_student(is_passing, average_grade, 75, 45, 95); // returns "U"
grade_student(is_passing, average_grade, 50, 75, 65); // returns "G"
grade_student(is_passing, average_grade, 80, 75, 95); // returns "VG"
```

Answer:

```
const is_passing = x => x >= 50;
const average_grade = (x, y, z) => (x + y + x)/3 < 75 ? "G" : "VG";
```

Question 6: List principle (2 points)

What is printed by the two calls to `display_list` in the following program?

```
const ls = list(3, 7, 5);
display_list(tail(tail(ls)));
display_list(pair(tail(ls), pair(9, null)));
```

Write down the answer using **list notation**, i.e. use the notation `list(...)` for any structure that satisfies the list principle (i.e. is a list as defined in the lecture).

```
list(5)
list(list(7, 5), 9)
```

Question 7: Function specifications (3 points)

Write a **full** function specification for the following function, including **at least two examples** that explain the behavior of the function:

```
function lcp(list1, list2) {
    return is_null(list1) || is_null(list2)
                ? null
            : head(list1) !== head(list2)
                ? null
            : pair(head(list1), lcp(tail(list1), tail(list2)));
}
```

Answer:

```
/**
 * Computes the longest common prefix of two lists.
 * @example
 * lcp(list(1, 2, 3, 4), list(1, 2, 3, 6))  // returns list(1, 2, 3)
 * lcp(list(1, 2, 3, 4), list(3, 2, 3, 6))  // returns list()
 * @param {list} list1 - The first list
 * @param {list} list2 - The second list
 * @returns The list of elements that both list1 and list2 begin with.
 */
```

Question 8: Variants (2 points)

Give a variant for the function `lcp` from the previous Question.

Answer: `length(list1)`

Answer: `length(list1) + length(list2)`

Justify your answer:
The length of a list is always a nonnegative integer, and in the single recursive call the lengths of both lists decrease by one. Hence `length(list1)` satisfies both conditions of a variant.

Question 9: List processing abstractions (3 points)

The function `average_above` takes as input a list of numbers `lst` and an integer `threshold`:

```
function average_above(lst, threshold) {
    return accumulate((acc, x) => acc + x,
                      0,
                      filter(x => x >= threshold, lst));
}
```

What is the result of the following code?

```
const lst = list(2, 5, 7, 1, 3, 0, 3, 7, 3);
average_above(lst, 5);
```

Answer: 19

Rewrite the function `average_above` using the higher-order abstraction functions `accumulate` and `map`, but **without** `filter`.
Answer:

```
function average_above2(lst, threshold) {
    return accumulate((acc, x) => acc + x,
                      0,
                      map(x => x >= threshold ? x : 0, lst));
}
```

Question 10: Memoization (2 points)

Is the function `average_above` from the previous Question a good candidate for memoization?
Explain your answer.
Answer:
No, this function is not a good candidate. There are no expensive repeated computations, and thus the function will not benefit from memoization that would store intermediate results to retrieve them without re-computation.

Question 11: Trees (3 points)

Recall the binary tree interface with the functions `make_empty_tree()`, `is_empty_tree(tree)`, `make_tree(value, left, right)`, `right_branch(tree)`, `left_branch(tree)`, `entry(tree)`.
Write a function `min_value` that takes as input a **binary search tree** constructed using the binary tree interface, and returns the smallest value in the tree.
You should assume that the input tree is non-empty, and all values in the tree are integers.
Your function should be as efficient as possible and **avoid any unnecessary computation**.
Answer:

```
function min_value(tree) {
    return is_empty_tree(left_branch(tree))
                ? entry(tree)
            : min_value(left_branch(tree));
}
```

Question 12: Environment model (3 points)

What are the values of `a` and `b` at the end of the following code?

```
let a = 3;
2 let b = 5;

4 function foo(b, c) {
      a = b;
6     b = c;
  }
8 foo(4, 7);

10 // values of a and b here?
```

Answer: 4 and 5

Explain your answer **using the environment model**:
The global variables a and b are stored in the program environment. The function call foo(4, 7) creates one new frame, which stored two variables b and c, that initially have the value 4 and 7, respectively.
The assignment on line 5 modifies the variable a that is located in the enclosing (program) environment. The assignment on line 6 modifies the local variable b located in the function environment (not the program environment).
Hence, only the variable a is modified outside of function foo.

Question 13: Arrays (2 points)

What is the contents of the array `arr` at the end of this code?

```
const arr = [[1, 2, 3],
             [3, 5, 2],
             [3, 6, 2]];

function add(A) {
    for(let i = 0; i < array_length(A); i = i + 1) {
        let x = 0;
        let j = 0;
```

```
        for(j = 0; j < array_length(A[i]); j = j + 1) {
            x = x + A[i][j];
        }
        A[i][j] = x;
    }
}

add(arr);
// contents of arr here?
```

```
[[1, 2, 3, 6],
 [3, 5, 2, 10],
 [3, 6, 2, 11]]
```

Question 14: Sorting (2 points)

Recall the **Selection Sort** sorting algorithm for sorting values in increasing order and its **in-place** implementation using arrays. Show the contents of the array as it changes during the run of Selection Sort after each step of the algorithm, i.e. after each step of the outer loop of the algorithm.

Answer:

14	5	12	7	9
5	14	12	7	9
5	7	12	14	9
5	7	9	14	12
5	7	9	12	14

Question 15: Streams (3 points)

Write a function `blank_every_third` that takes as input an **infinite stream** without side-effects and returns a stream that inserts a '0' into every third position. For example, recall the infinite streams of 1's (`ones`) and integers (`integers`). Applying `blank_every_third` to them will return the following streams:

```
// 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, ...
blank_every_third(ones);

// 1, 2, 0, 3, 4, 0, 5, 6, 0, 7, 8, 0, 9 ...
blank_every_third(integers);
```

Your implementation does not have to be particularly optimized.
Answer:

```
function blank_every_third(stream) {
    return pair(head(stream),
                () => pair(head(stream_tail(stream)),
                           () => pair(0,
                                      () =>
                        blank_every_third(stream_tail(stream_tail(stream))))));
}
```

Question 16: Development methodology (2 points)

In this question we will extend a course registration system for a university. As in Homework 6, students (modelled as `strings`) can apply to take various courses. Each course has a fixed number of slots, and if the course is full there may also be students on a waitlist ordered by their application time. You can assume, if needed, that the accessor functions `name(course)`, `credits(course)`, `admitted(course)`, `waitlist(course)` exist.
One problem is that students who are already admitted to courses may remain on the waitlists of other courses. Therefore, we are investigating how to write a function that given a student and a list of courses, checks if they are already admitted to courses adding up to the maximum number of credits (45), and if so, removes the student from all waitlists.
Begin implementing this function using **bluffing** as one of the first steps. You do not need to develop your solution further than to complete the bluffing step.

```
function find_and_remove_if_oversubscribed(student) {
    const admitted_points = points_of_student(courses, student);
    if (admitted_points >== 45) {
        remove_from_waitlists(courses, student);
    } else {}
}
// Bluff: adds up the total number of points the student is admitted to among the courses.
function points_of_student(courses, student) {}
// Bluff: removes the student from the waitlists of all of courses.
function remove_from_waitlists(courses, student) {}
```

This is the final page.